Types

Basic types

Basic Types

`let a: string = "hi"`	text
`let n: number = 5`	any number
`let ok: boolean = true`	true / false
`let arr: number[] = [1, 2]`	array
`let t: [string, number]`	tuple
`let u: undefined  let v: null`	empty types
`let big: bigint = 1n`	big number

Special Types

`any`	turn off checks
`unknown`	safe any
`void`	no return value
`never`	never happens
`object`	non-primitive

Union & Literal

`let id: string | number`	either type
`let d: “up” | "down"`	exact values
`x as string`	type assertion
`x!`	non-null assertion
`typeof x  keyof T`	type from value / keys

Arrays, Tuples & Objects

Shape your data

Arrays & Tuples

`let a: number[]`	array of numbers
`let a: Array<string>`	generic array form
`let r: readonly number[]`	cannot change
`let t: [string, number]`	fixed tuple
`let t: [string, number?]`	optional tuple item
`let t: [...number[], string]`	rest in tuple

Object Types

`let o: { x: number; y?: string }`	inline object type
`let o: { readonly id: number }`	read-only field
`let d: Record<string, number>`	key-value object
`let d: { [k: string]: number }`	index signature
`let f: { (a: number): void }`	callable object

Helpers

`as const`	freeze literal types
`satisfies T`	check without widening
`typeof arr[number]`	item type of array
`keyof typeof obj`	keys as a type
`Object.keys(o) as (keyof T)[]`	typed keys

Literal & Template Types

Exact values

Literal Types

`let d: "left" | "right"`	one of strings
`let n: 1 | 2 | 3`	one of numbers
`let b: true`	only true
`const x = "a"`	const gives a literal type
`type Dir = "up" | "down"`	named literal union

Template Literal Types

`type Id = \`id_${number}\``	pattern string
`type Ev = \`on${Capitalize<string>}\``	built from parts
`Uppercase<"a">`	"A"
`Lowercase<"A">`	"a"
`Capitalize<"abc">`	"Abc"
`Uncapitalize<"Abc">`	"abc"

Enums & Unions

`enum C { Red, Green }`	numeric enum
`enum C { Red = "R" }`	string enum
`const enum C { A }`	inlined enum
`type K = keyof typeof C`	enum names as a type
`Object.values(C)`	enum values

Interfaces & Aliases

Shape your data

Interface

`interface User { name: string }`	object shape
`age?: number`	optional property
`readonly id: number`	cannot change
`[key: string]: any`	index signature
`interface B extends A { }`	extend

Type Alias

`type ID = string | number`	alias
`type P = { x: number }`	object type
`type AB = A & B`	intersection
`type Fn = (a: number) => void`	function type

Enums

`enum Dir { Up, Down }`	numeric enum
`enum C { Red = "RED" }`	string enum
`const enum E { A }`	inlined enum
`as const`	literal types

Generics Deep Dive

Flexible types

Constraints

`<T extends string>`	must be a string
`<T extends { id: number }>`	must have an id
`<K extends keyof T>`	key of another type
`<T = string>`	default type
`<T extends unknown[]>`	array type

Using Generics

`function f<T>(x: T): T`	generic function
`const f = <T,>(x: T) => x`	generic arrow (TSX)
`class Box<T> { v: T }`	generic class
`interface P<T> { v: T }`	generic interface
`f<number>(5)`	explicit type

Inference Helpers

`ReturnType<typeof f>`	function result type
`Parameters<typeof f>`	argument tuple
`Awaited<Promise<T>>`	unwrap promise
`infer U`	capture a type
`NoInfer<T>`	do not infer from here

Functions & Generics

Typed functions

Functions

`function f(a: number): number { }`	typed function
`const f = (a: number): void => { }`	arrow function
`function f(a?: number)`	optional param
`function f(a = 5)`	default param
`function f(...r: number[])`	rest params
`function f(this: T)`	this type

Overloads

`function f(a: string): string;`	overload signature
`function f(a: number): number;`	another signature

Generics

`function f<T>(x: T): T { }`	generic function
`interface Box<T> { v: T }`	generic interface
`<T extends { id: number }>`	constraint
`<T = string>`	default type

Narrowing

`typeof x === "string"`	type guard
`x instanceof Date`	class guard
`"a" in obj`	property guard
`function isA(x): x is A`	custom guard

Mapped & Conditional

Type-level logic

Mapped Types

`{ [K in keyof T]: T[K] }`	copy a type
`{ [K in keyof T]?: T[K] }`	make all optional
`{ readonly [K in keyof T]: T[K] }`	make read-only
`{ -readonly [K in keyof T]-?: T[K] }`	remove modifiers
`{ [K in keyof T as `get${K}`]: T[K] }`	rename keys

Conditional Types

`T extends U ? X : Y`	if-else for types
`T extends any[] ? T[number] : T`	unwrap arrays
`T extends string ? "s" : "n"`	pick by type
`Exclude<T, U>`	remove from union
`Extract<T, U>`	keep from union
`NonNullable<T>`	remove null and undefined

Distribution

`type A<T> = T extends any ? T[] : never`	per union member
`[T] extends [U]`	stop distribution
`T[number]`	item type
`T["key"]`	indexed access

Type Guards & Assertions

Narrow types

Built-in Guards

`typeof x === "string"`	primitive check
`x instanceof Date`	class check
`"id" in obj`	property check
`Array.isArray(x)`	array check
`x === null`	null check
`x != null`	not null or undefined

Custom Guards

`function isA(x: any): x is A`	type predicate
`function check(x): asserts x is A`	assertion function
`function assert(c: unknown): asserts c`	assert truthy
`if (isA(x)) { x }`	narrowed inside

Assertions

`x as string`	trust me
`x as unknown as T`	force a cast
`<string>x`	old cast style
`x!`	not null assertion
`x satisfies T`	validate shape

Classes

OOP with types

Classes

`class A { x: number }`	class with field
`constructor(public x: number) { }`	shorthand fields
`private: protected: public:`	access
`readonly id: string`	set once
`static count = 0`	class property
`get v() { }  set v(x) { }`	accessors

Inheritance

`class B extends A { }`	inherit
`class C implements I { }`	follow interface
`abstract class A { }`	cannot instantiate
`abstract f(): void`	must override
`super(x)`	call parent
`override f() { }`	safe override

Decorators & Modules

`@Component({ })`	decorator
`export  import`	modules
`import type { T } from "./t"`	type-only import
`namespace N { }`	group code

Utility Types

Built-in helpers

Object Utilities

`Partial<I>`	all optional
`Required<T>`	all required
`Readonly<I>`	all readonly
`Pick<T, "a" | "b">`	keep keys
`Omit<T, "a">`	remove keys
`Record<string, number>`	key-value map

Other Utilities

`Exclude<U, "a">`	remove from union
`Extract<U, "a">`	keep in union
`NonNullable<T>`	no null / undefined
`ReturnType<typeof f>`	function result
`Parameters<typeof f>`	param tuple
`Awaited<Promise<T>>`	unwrap promise

Advanced

`T extends U ? X : Y`	conditional type
`{ [K in keyof T]: T[K] }`	mapped type
`` `${A}-${B}` ``	template literal type
`infer R`	capture a type

Async Types

Promises typed

Promises

`Promise<string>`	future string
`async function f(): Promise<number>`	async return type
`await p`	unwrap promise
`Promise.all([a, b])`	wait for all (tuple)
`Promise.allSettled(ps)`	all outcomes
`Promise.race(ps)`	first to finish

Errors

`catch (e: unknown)`	catch is unknown
`e instanceof Error`	narrow the error
`class MyErr extends Error { }`	custom error
`type R<T> = Ok<T> | Err`	result type

Streams & Fetch

`const r = await fetch(url);`	fetch response
`const d = (await r.json()) as T;`	typed JSON
`AsyncGenerator<number>`	async generator type
`for await (const x of it) { }`	async loop
`AbortController`	cancel requests

Modules & Declarations

Imports and .d.ts

Import & Export

`import { a } from "./a";`	named import
`import type { T } from "./a";`	type-only import
`import x, * as y from "./a";`	default + namespace
`export type { T };`	export a type
`export * from "./a";`	re-export all

Declarations

`declare const X: string;`	describe a global
`declare module "lib" { }`	type a module
`declare global { }`	extend globals
`*.d.ts`	declaration file
`npm i -D @types/node`	types for a library

Config & Paths

`"moduleResolution": "bundler"`	how imports resolve
`"paths": { "@/*": ["src/*"] }`	import aliases
`"verbatimModuleSyntax": true`	keep import types clear
`/// <reference types="node" />`	include types
`namespace N { }`	old grouping style

DOM & Node Types

Browser and server

DOM Types

`querySelector<HTMLInputElement>("#a")`	typed element
`HTMLElement  HTMLButtonElement`	element types
`e: MouseEvent  e: KeyboardEvent`	event types
`e.target as HTMLInputElement`	typed target
`el?.addEventListener("click", fn)`	safe listener

Node Types

`import fs from "node:fs";`	Node module
`process.env.KEY`	string | undefined
`Buffer  NodeJS.Timeout`	common Node types
`req: Request, res: Response`	Express handler types

React Types

`React.FC<Props>`	function component
`useState<number>(0)`	typed state
`React.ReactNode`	any renderable
`React.ChangeEvent<HTMLInputElement>`	input event
`React.ComponentProps<"button">`	reuse props

Strict Mode & Errors

Catch bugs early

Strict Options

`"strict": true`	turn on all checks
`"noImplicitAny": true`	no hidden any
`"strictNullChecks": true`	null must be handled
`"noUnusedLocals": true`	flag unused variables
`"exactOptionalPropertyTypes"`	optional means missing
`"noUncheckedIndexedAccess"`	array items may be undefined

Comments & Escapes

`// @ts-ignore`	hide next error
`// @ts-expect-error`	expect next error
`// @ts-nocheck`	skip this file
`unknown over any`	safe unknown value
`never`	should be unreachable

Common Errors

`TS2322`	type not assignable
`TS2339`	property does not exist
`TS2345`	wrong argument type
`TS7006`	implicit any parameter
`TS2532`	object possibly undefined

Tips & tsc

Config and habits

tsc

`npm i -D typescript`	install
`npx tsc --init`	make tsconfig
`npx tsc`	compile project
`npx tsc --watch`	rebuild on save
`npx tsx file.ts`	run directly

tsconfig

`"strict": true`	best safety
`"target": "ES2022"`	output version
`"outDir": "dist"`	output folder
`"noUncheckedIndexedAccess"`	safer indexing

Good Habits

`Avoid any, use unknown`	safer types
`Turn on strict mode`	catch bugs early

Keep coding. Keep learning.

Made by

@hafizibrahimsuriya

Save this sheet | Share it | Practice daily